FIAP - Pós Tech Arquitetura e Desenvolvimento Java (ADJT)
Tech Challenge - Fase 4

Plataforma de Feedback de Aulas
Documento de Arquitetura

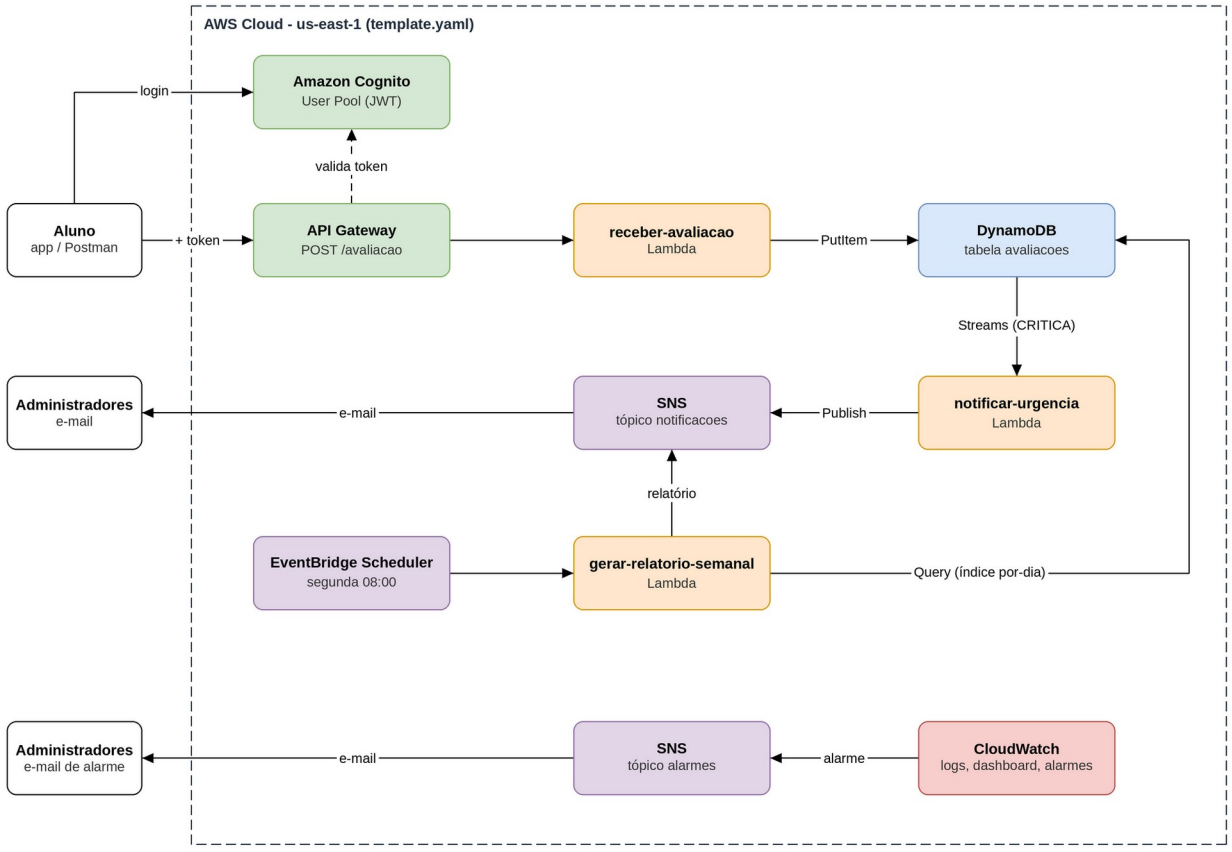
Grupo: Igor, Caio e Luciano
Repositório: https://github.com/Igolbr/cloud-tech-challenge_fase_4
Novembro de 2026

1. Objetivo

Este documento descreve a arquitetura da plataforma de feedback de aulas: qual modelo de nuvem escolhemos, os componentes usados, como eles se comunicam, as dependências e as configurações de segurança do ambiente. As instruções de deploy e a documentação de cada função estão no README do repositório.

Resumo do problema: o aluno avalia a aula com uma descrição e uma nota de 0 a 10. Quando a avaliação é crítica, os administradores precisam ser avisados por e-mail na hora. Toda semana eles também recebem um relatório com a média das notas, a quantidade de avaliações por dia e por urgência.

2. Desenho da arquitetura



Todos os recursos dentro da nuvem são criados pelo template.yaml (AWS SAM) via GitHub Actions.

Fluxo de uma avaliação crítica:

1. O aluno faz login no Cognito com usuário e senha e recebe um token JWT (vale 1 hora).
2. O aluno envia `POST /avaliacao` com `descricao` e `nota` e o cabeçalho `Authorization: Bearer <token>`. O API Gateway valida o token antes de chamar a função; sem token válido ele

responde 401.
3. A função **receber-avaliacao** valida o corpo, calcula a urgência pela nota e grava no DynamoDB. O aluno recebe 201.
4. A gravação gera um registro no DynamoDB Streams. O gatilho tem um filtro que só deixa passar inserções com urgência CRITICA, e aí a função **notificar-urgencia** é chamada.
5. A notificar-urgencia publica no tópico SNS e os administradores recebem o e-mail com descrição, urgência e data de envio.
6. Na segunda-feira às 08:00 o EventBridge Scheduler chama a **gerar-relatorio-semanal**, que consulta os 7 dias anteriores e envia o relatório pelo mesmo tópico.

Uma função nunca chama a outra diretamente. Elas são ligadas por eventos da AWS (API Gateway, Streams e Scheduler). Com isso, se o envio do e-mail falhar, o aluno não recebe erro, porque a avaliação já foi gravada e o envio é tentado de novo.

3. Modelo de nuvem escolhido

Modelo de implantação: nuvem pública (AWS). Tudo roda na AWS, na região us-east-1. Não temos servidor próprio nem parte on-premises.

Modelo de serviço: serverless. O código das regras de negócio roda como FaaS (Function as a Service) no AWS Lambda. O restante (banco, mensageria, agendamento, autenticação e API) são serviços gerenciados, em que a AWS cuida de servidor, escala, disponibilidade e atualização, e a gente só configura.

Infraestrutura como código. Todos os recursos estão no arquivo `template.yaml` (AWS SAM, que é convertido em CloudFormation). Com um `sam deploy` o ambiente inteiro é criado ou atualizado, sem precisar configurar nada pelo console.

Por que serverless

- O uso é esporádico: os alunos avaliam depois das aulas e o relatório roda uma vez por semana. Um servidor ligado o tempo todo ficaria parado a maior parte do tempo. Com Lambda e DynamoDB on-demand o custo acompanha o uso e, no volume do projeto, fica na camada gratuita.
- O problema é orientado a eventos (chegou avaliação, grava; gravou uma crítica, avisa; chegou segunda, consolida), então cada evento vira o gatilho de uma função.
- Não tem sistema operacional nem patch para manter, o que ajuda já que os créditos de nuvem são limitados.

Por que AWS

Comparamos os serviços equivalentes nas três nuvens principais:

Necessidade	AWS (escolhida)	Azure	GCP
Funções	Lambda	Functions	Cloud Functions / Cloud Run
Banco NoSQL serverless	DynamoDB on-demand	Cosmos DB	Firestore
Agendamento	EventBridge Scheduler	Timer Trigger	Cloud Scheduler
E-mail sem domínio próprio	SNS com assinatura de e-mail	Precisa de domínio ou Logic Apps	Não tem nativo (SendGrid etc.)
Autenticação da API	Cognito + JWT authorizer	Entra ID / API Management	Identity Platform / API Gateway
Infraestrutura como código	SAM (um template YAML)	Bicep / ARM	Terraform ou gcloud

O que mais pesou foi o e-mail: com o SNS o administrador só precisa confirmar a assinatura, sem

comprar domínio nem usar serviço de terceiros. Também ajudou o SAM permitir declarar funções, API, tabela, alarmes e permissões num arquivo só, o que deixa o deploy automático mais simples.

4. Componentes

Os nomes reais recebem o ambiente no final (por exemplo `receber-avaliacao-dev`).

Componente	Serviço AWS	Configuração	Papel na solução
Identidade	Cognito User Pool e App Client	Cadastro só pelo administrador; senha com no mínimo 12 caracteres; token de 1 hora	Autentica o aluno e emite o JWT exigido pela API
API	API Gateway HTTP API	Rota POST /avaliacao; JWT authorizer; limite de 10 req/s (picos de 20); log de acesso sem IP	Entrada pública da aplicação, só HTTPS
Funções	AWS Lambda	Java 21, arm64, 512 MB; timeout de 30 s (60 s no relatório)	Regras de negócio, uma responsabilidade por função
Banco de dados	DynamoDB	On-demand; chave id; índice por-dia; Streams; criptografia KMS; backup contínuo de 35 dias	Guarda as avaliações e dispara o aviso de urgência
Notificações	SNS (tópico notificacoes)	Assinatura de e-mail do administrador; nega publicação sem TLS	Envia o e-mail de urgência e o relatório
Agendamento	EventBridge Scheduler	cron(0 8 ? * MON *) no fuso America/Sao_Paulo; 2 novas tentativas	Dispara o relatório semanal
Monitoramento	CloudWatch	Logs por 30 dias; dashboard; 9 alarmes; 2 filtros de métrica nos logs	Acompanha funções, API e tabela
Alarmes	SNS (tópico alarmes)	Separado do tópico de negócio	Avisa os administradores sobre falhas técnicas
Governança	IAM, CloudTrail, Budgets	Um papel por função; trilha em todas as regiões; orçamento de US$ 10/mês	Menor privilégio, auditoria e controle de custo
Entrega	GitHub Actions e CloudFormation	Testes, lint e sam deploy a cada push na main; acesso por OIDC	Deploy automatizado

5. Funções serverless

O enunciado pede no mínimo duas funções com responsabilidade única. Fizemos três porque são três gatilhos diferentes: gravar (requisição do aluno), avisar (nova avaliação crítica) e consolidar (agendamento semanal). Juntar duas delas misturaria responsabilidades.

Função	Responsabilidade	Gatilho	Saída	Permissão IAM
receber-avaliacao	Validar e gravar a avaliação	POST /avaliacao (depois do JWT authorizer)	201 com o item gravado ou 400 com a lista de erros	dynamodb:PutItem na tabela
notificar-urgencia	Avisar sobre avaliações críticas	DynamoDB Streams, filtro INSERT e urgencia = CRITICA	E-mail via SNS com descrição, urgência e data de envio	Ler o stream; sns:Publish no tópico de notificações
gerar-relatorio-semanal	Consolidar a semana	EventBridge Scheduler (segunda 08:00) ou invocação manual	E-mail com média, quantidade por dia, por urgência e a lista das avaliações	dynamodb:Query no índice por-dia; sns:Publish

Tratamento de falhas: na receber-avaliacao, erro ao gravar responde 500 e dispara o alarme. Na notificar-urgencia usamos resposta parcial do lote, então só o registro que falhou é reprocessado (até 3 vezes) e nenhum e-mail sai repetido. Na gerar-relatorio-semanal o agendador tenta mais 2 vezes e dá para rodar a função manualmente.

6. Comunicação entre os serviços

Só o caminho do aluno até a gravação é síncrono. O aviso e o relatório são assíncronos.

Origem	Destino	Mecanismo	Modo	Se falhar
Aluno	Cognito	InitiateAuth (HTTPS)	Síncrono	Login recusado sem informar se o usuário existe
Aluno	API Gateway	HTTPS com Authorization: Bearer	Síncrono	401 sem token válido; 429 acima do limite
API Gateway	receber-avaliacao	Integração Lambda proxy (payload 2.0)	Síncrono	5xx para o aluno e alarme da API
receber-avaliacao	DynamoDB	PutItem (AWS SDK for Java v2)	Síncrono	500 e alarme de gravação
DynamoDB	notificar-urgencia	Streams com filtro no event source mapping	Assíncrono	Reprocessa só o registro com falha, até 3 vezes
notificar-urgencia	SNS e e-mail	Publish no tópico de notificações	Assíncrono	Alarme de falha de envio
EventBridge	gerar-relatorio-semanal	Agendamento semanal	Assíncrono	2 novas tentativas e alarme de erro
gerar-relatorio-semanal	DynamoDB	7 consultas (Query) no índice por-dia, sem Scan	Síncrono	Erro da função e alarme
CloudWatch	SNS alarmes e e-mail	Ação do alarme	Assíncrono	Gerenciado pela AWS

7. Modelo de dados

Tabela `avaliacoes-<ambiente>`. Não gravamos nenhum dado pessoal do aluno; a identidade dele fica só no Cognito.

Atributo	Tipo	Exemplo	Uso
id	texto (UUID)	6f1c2a52-...	Chave de partição
descricao	texto	O áudio da aula 3 está inaudível	Conteúdo da avaliação
nota	número	2	Inteiro de 0 a 10, usado na média
urgencia	texto	CRITICA	Calculada pela nota; usada no filtro do Streams
dataEnvio	texto ISO-8601 (UTC)	2026-11-03T17:02:11Z	Chave de ordenação do índice por-dia
dia	texto yyyy-MM-dd	2026-11-03	Chave de partição do índice por-dia (fuso de São Paulo)

Regra de urgência. O enunciado não diz o que é urgência, então definimos pela faixa da nota: 0 a 3 é CRITICA (manda e-mail na hora), 4 a 6 é MEDIA e 7 a 10 é BAIXA.

Índice por-dia. O relatório precisa das avaliações de 7 dias. Com o índice a função faz uma consulta por dia e lê só o que precisa. Sem ele teríamos que fazer um Scan na tabela inteira, que fica mais caro e lento conforme a tabela cresce e exigiria uma permissão de leitura maior.

8. Dependências

Componente	Depende de
receber-avaliacao	Cognito (pelo authorizer), API Gateway, DynamoDB
notificar-urgencia	DynamoDB Streams, SNS notificações
gerar-relatorio-semanal	EventBridge Scheduler, índice por-dia, SNS notificações

Alarmes	CloudWatch, SNS alarmes
Entrega dos e-mails	Assinatura confirmada pelo administrador

Software e versões: runtime Lambda java21 (arm64), AWS SDK for Java 2.29.0, aws-lambda-java-core 1.2.3, aws-lambda-java-events 3.14.0, Jackson 2.17.2 e JUnit 5.10.2. Build com Maven 3.9+ e AWS SAM CLI; CI/CD no GitHub Actions com cfn-lint.

9. Configuração do ambiente e segurança

Autenticação da API

- O POST /avaliacao exige um JWT emitido pelo Cognito. O API Gateway confere assinatura, emissor, público e validade antes de chamar a função. Sem token válido a resposta é 401 e nada é executado.
- Autocadastro desligado: só a equipe cria alunos. Recuperação de conta só pelo administrador.
- Token de acesso de 1 hora e refresh token de 1 dia, com revogação habilitada.

Proteção dos dados

- Criptografia em repouso no DynamoDB com chave KMS gerenciada pela AWS e backup contínuo de 35 dias.
- Tráfego só em HTTPS: a HTTP API não tem endpoint HTTP e os tópicos SNS negam publicação sem TLS.
- Nenhum dado pessoal na avaliação; o log de acesso da API não registra IP nem user agent.
- Limite de 10 requisições por segundo (picos de 20) para evitar abuso.

Permissões das funções (menor privilégio)

Papel IAM	Pode	Não pode (exemplos)
receber-avaliacao	PutItem na tabela; escrever no próprio log	Ler, alterar ou apagar avaliações; publicar no SNS
notificar-urgencia	Ler o stream; publicar no tópico de notificações; escrever no próprio log	Ler ou gravar na tabela; publicar no tópico de alarmes
gerar-relatorio-semanal	Query no índice por-dia; publicar no tópico de notificações; escrever no próprio log	Scan, gravar ou apagar na tabela; publicar no tópico de alarmes

O script `scripts/verificar-permissoes.sh` usa o simulador de políticas do IAM para comprovar isso.

Governança da conta

- O GitHub Actions acessa a AWS por OIDC, com token temporário; nenhuma chave fica guardada.
- O CloudFormation usa um papel de execução próprio.
- CloudTrail em todas as regiões, com validação de integridade, para auditoria.
- Usuários IAM nominais para a equipe, com MFA obrigatório. A conta raiz não é usada no dia a dia.
- Custo: tags Projeto e Ambiente em todos os recursos, orçamento mensal de US$ 10 com aviso por e-mail e logs guardados por 30 dias.

10. Monitoramento

O dashboard `plataforma-feedback-<ambiente>` no CloudWatch mostra o estado dos alarmes, invocações, erros, duração e throttles das funções, requisições, erros 4xx/5xx e latência da API, o consumo da tabela e as falhas mais recentes nos logs. Os alarmes mandam e-mail pelo tópico SNS de alarmes:

Alarme	Dispara quando

Erros de cada função (3 alarmes)	1 ou mais erros em 5 minutos
Duração de cada função (3 alarmes)	A execução passa de 80% do timeout
API 5xx	Algum aluno recebeu erro de servidor
Gravação no DynamoDB	Falha da aplicação, erro de sistema ou throttling da tabela
Envio do aviso de urgência	O SNS recusou o aviso de uma avaliação crítica

Para testar, o script `scripts/forcar-erro.sh` provoca um erro na receber-avaliacao e o e-mail de alarme chega em até 5 minutos.

11. Deploy automatizado

1. Em todo push e pull request o GitHub Actions roda `mvn verify` (testes das três funções) e o cfn-lint nos templates.
2. Em push na main ele pega credenciais temporárias por OIDC (ou, enquanto o OIDC não estiver configurado, usa um usuário de deploy com as chaves nos secrets do repositório).
3. Roda `sam build` e `sam deploy`, que aplica as mudanças na pilha pelo CloudFormation.
4. No final o pipeline mostra a URL da API e do dashboard.

O passo a passo para implantar do zero está no README do repositório.